LAB: USE CASE MODELING

Requirements Analysis and UML

Course	Software Requirements / SRE	**Lab No.**	05
Student Name	_____	**Date**	Oct 6th-26
Section	A	**Duration**	90 minutes

1. Lab Overview

Use Case Modeling is a requirements-analysis technique used to describe how external actors interact with a system to achieve meaningful goals. In this lab, you will analyze a real-world system scenario, identify actors and their goals, establish relationships, and construct a UML Use Case Diagram.

2. Learning Objectives

- Identify actors that interact with a system.
- Distinguish actors from ordinary system components or internal processes.
- Identify meaningful user goals and convert them into candidate use cases.
- Define and draw the system boundary.
- Represent associations between actors and use cases.
- Understand when include, extend, and generalization relationships may be appropriate.
- Construct a clear UML Use Case Diagram using a modeling tool such as draw.io/Star UML etc.
- Review a use case model for completeness, consistency, and clarity.

3. Core Concepts to Understand Before the Lab

Concept	Meaning
Actor	An external entity that interacts with the system to achieve a goal. An actor may be a person, another system, or an external device/service.
Use Case	A meaningful goal or service that an actor wants to accomplish through the system. Examples include Register Course, Search Course, or View Timetable.
System Boundary	A boundary that defines what belongs to the system being modeled. Use cases are inside the boundary; external actors are outside it.
Association	A relationship showing that an actor participates in or interacts with a use case.
«include»	Used when a behavior is required as part of another use case. Think: the included behavior must occur.
«extend»	Used when additional behavior occurs conditionally or optionally in the context of a base use case.
Generalization	Represents a specialized actor or use case inheriting characteristics or behavior from a more general one.

4. A Professional Modeling Mindset

Do not begin by asking: "What functions or buttons does the software have?"

Instead ask: "Who interacts with the system, and what meaningful goal are they trying to accomplish?"

For example, “Click Register Button” is normally a user-interface action rather than a meaningful use case. “Register for Course” expresses a user goal and is therefore a stronger candidate use case.

5. Lab Scenario: University Course Registration System

A university is developing an online Course Registration System. Students use the system to log in, search for available courses, view course information and available seats, register for courses, drop courses, and view their timetable. A Registrar manages course information and registration rules. Some courses may require a registration fee, which is handled through an external payment service.

6. Task 1 - Identify the System Boundary

Determine exactly what system is being modeled.

- Write the name of the system.
- State what is inside the system boundary.
- Identify entities that are outside the system boundary.

Deliverable: A labeled system boundary sketch.

7. Task 2 - Identify Actors

Read the scenario and identify all potential actors.

- List each actor.
- For every actor, explain why it is external to the system.
- Determine whether each actor is a human, external software system/service, or device.

Use the following table in your report:

Actor	Type	Reason for Interaction

8. Task 3 - Identify Actor Goals / Candidate Use Cases

For each actor, determine what meaningful goals they want to accomplish through the system.

Prepare an Actor–Goal table:

Actor	Candidate Goal / Use Case

9. Task 4 - Draw the Basic Use Case Diagram

Using Star UML, Eclipse Papyrus, Mermaid or another UML modeling tool, create the first version of your Use Case Diagram.

- Draw the system boundary.
- Place actors outside the boundary.
- Place use cases inside the boundary.
- Connect actors to relevant use cases using associations.
- Use meaningful names for actors and use cases.
- Keep the diagram readable and avoid unnecessary crossing lines.

10. Task 5 - Analyze «include» and «extend»

Review your use cases and look for relationships between them.

For «include», ask: "Is this behavior required every time the base use case is performed?"

For «extend», ask: "Can the base use case complete without this additional behavior, with the extra behavior occurring only under a condition?"

- Identify at least one possible «include» relationship if the scenario supports one.
- Identify at least one possible «extend» relationship if the scenario supports one.
- If you believe no relationship is justified, explain why rather than inventing one.
- Use correct UML direction and labels.

11. Task 6 - Consider Generalization

Examine the actors and use cases for possible generalization. Do not force a generalization relationship merely to make the diagram more complex.

- Determine whether any actor is a specialized form of another actor.
- If applicable, draw the generalization relationship and justify it.
- If not applicable, state why generalization is unnecessary in this scenario.

12. Task 7 - Review Your Model Like a Requirements Analyst

Before submission, check your model against the following questions:

- ☐ Is the system boundary clearly defined?
- ☐ Are actors outside the system boundary?
- ☐ Are use cases inside the system boundary?
- ☐ Does every use case represent a meaningful goal?
- ☐ Can you explain why every actor interacts with the system?
- ☐ Have you confused user-interface actions with use cases?
- ☐ Are «include» relationships truly mandatory/reusable behavior?
- ☐ Are «extend» relationships truly conditional/optional behavior?
- ☐ Are relationship directions and labels correct?
- ☐ Is the diagram readable without excessive crossing lines?
- ☐ Have you avoided unnecessary or duplicate use cases?

13. Task 8 - Write One Use Case Description

Select one important use case from your diagram and document it using the following structure.

Use Case Name	
Primary Actor	
Goal	
Preconditions	
Main Success Scenario	1. _____ 2. _____ 3. _____ 4. _____ 5. _____
Alternative / Exception Flows	_____ _____
Postconditions	

14. Lab Reflection Questions

1. What is the difference between an actor and a user?
2. Why must actors normally be outside the system boundary?
3. Why is “Register for Course” a stronger use case candidate than “Click Register Button”?
4. What is the practical difference between «include» and «extend»?

5. What mistake would occur if every system function were treated as a separate use case?
6. How does a Use Case Diagram differ from a fully dressed Use Case?
7. What information does a Use Case Diagram intentionally leave out?

19. Key Takeaway

Use Case Modeling is about understanding WHO interacts with the system, WHAT meaningful goals they want to achieve, and HOW those interactions are represented at a high level.